MolGENT: Conditional Transformer-GAN–RL Driven Property-Aligned Molecular Generation

P. Dhanush[1], A. Smaran Reddy[1], Amod Adarsh[1], I. Siva Prasad Rao[1]
[1]Indian Institute of Information Technology, Raichur
Email: {cs23b1053, cs23b1011, cs23b1005, cs23b1027}@iiitr.ac.in

***Abstract*—Generative modeling of chemical structures using deep learning has become a promising strategy for molecular discovery. In this work, we present a complete pipeline combining dataset preprocessing, property computation, vocabulary construction, conditional Transformer-based sequence modeling, adversarial training, and reinforcement-learning (RL) driven fine-tuning. The core contribution of this pipeline is the conditional generator, which integrates molecular property vectors directly into the Transformer to produce property-aligned molecules. We show that the conditional model significantly outperforms the unconditional generator in validity and property compliance. Real evaluation results demonstrate that our conditional generator achieves 80% to 86% validity and strong alignment with desired chemical properties. This highlights the effectiveness of conditioning mechanisms combined with adversarial and RL training.**

I. Introduction

Deep generative models have demonstrated strong potential in de novo molecular design. While unconditional models can generate syntactically valid SMILES, they struggle to control chemical properties such as logP or SAS. Conditional molecular generation aims to address this limitation by guiding the generator using explicit property vectors.

This work proposes a conditional Transformer generator embedded in a GAN-to-RL training pipeline. Our focus is on demonstrating how conditioning improves validity and property alignment compared to unconditional generation. The contributions of this work include:

- A complete molecular preprocessing and property computation pipeline.
- A conditional Transformer generator that integrates property embeddings.
- Adversarial training with a property-aware discriminator.
- RL fine-tuning to directly optimize chemical-objective reward signals.
- Extensive sampling and real evaluation showing strong conditional performance.

II. Dataset Preprocessing

Raw SMILES data undergoes a systematic cleaning pipeline to ensure consistency and model readiness.

A. Cleaning and Canonicalization

RDKit was used to validate molecules, canonicalize SMILES, remove salts, and eliminate disconnected fragments. Duplicate removal ensures better training diversity.

B. Dataset Split

The processed dataset is divided into:

- 90% training
- 5% validation
- 5% testing

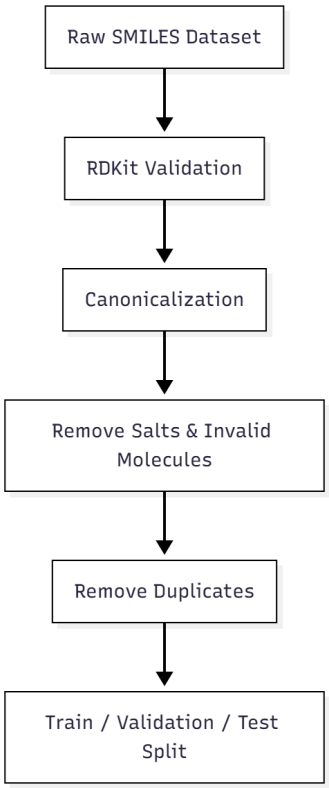

Fig. 1. Preprocessing workflow for molecular SMILES datasets.

III. Property Calculation

Molecular properties essential for conditional generation were computed using RDKit. These include:

- QED – Quantitative Estimate of Drug-likeness
- SAS – Synthetic Accessibility Score
- LogP – Partition Coefficient (octanol/water)
- TPSA – Topological Polar Surface Area
- MW – Molecular Weight

Property normalization ensures stable training for the conditional generator.

IV. Model Architecture

This section describes the two major components of our molecular design framework: the Conditional Transformer Generator and the Property-Aware Discriminator. Both models work with tokenized SMILES sequences and normalized molecular property vectors such as molecular weight, LogP, and synthetic accessibility scores. While the generator produces new molecules conditioned on desired properties, the discriminator evaluates both the realism and property alignment of these molecules.

A. Transformer Generator

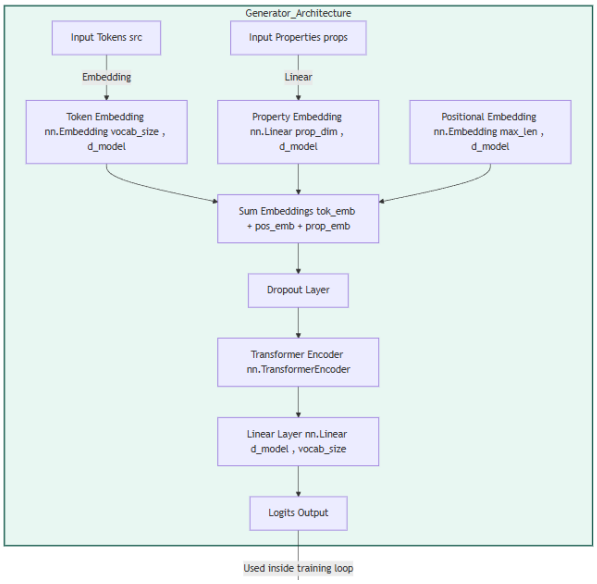

Fig. 2. Generator architecture

The generator is based on a Transformer decoder architecture tailored for autoregressive SMILES generation. Its role is to produce one token at a time until a complete, chemically valid SMILES string is formed. Unlike an unconditional language model, the generator is explicitly conditioned on a property vector, allowing it to bias generation toward molecules that satisfy specific chemical requirements.

1) Input Representation: The generator receives two kinds of inputs: a SMILES prefix (the tokens generated so far) and a numerical property vector. The SMILES tokens are first converted into embeddings, and a positional encoding is added so the model knows the order of characters. This helps the generator understand chemical structures such as ring closures and branch points.

2) Property Embedding and Conditioning Mechanism: The property vector is passed through a small neural network to transform it into the same feature space as the token embeddings. This processed property representation is then added to every token embedding in the input sequence. By injecting the property information at every position, the generator ensures that the target molecular characteristics influence every decoding step. This design makes the model strongly responsive to conditioning and enables meaningful control over the generated molecule's behavior.

3) Transformer Decoder Layers: The generator consists of several stacked decoder blocks, each containing multi-head self-attention and a feed-forward sublayer. The attention mechanism allows the model to look back at previously generated tokens and capture long-range chemical dependencies, such as aromatic ring structures, stereochemistry, and multi-atom functional groups. Residual connections and normalization help stabilize training and preserve gradient flow.

4) Token Prediction and Sampling: After passing through all decoder layers, the output features are projected into a probability distribution over the vocabulary of SMILES tokens. The generator then samples the next token, appends it to the sequence, and continues this process autoregressively until the end-of-sequence token is produced. Because the model is conditioned on properties, it tends to generate molecules whose predicted properties match the target profile.

B. Property-Aware Discriminator

The discriminator plays a crucial role in adversarial training and reinforcement learning. It not only determines whether a molecule looks realistic and chemically meaningful, but also checks whether the generated molecule aligns with the specified property conditions. This dual functionality makes the discriminator both a critic and a property evaluator.

1) SMILES Encoding: The discriminator takes a complete SMILES sequence—either real or generated—and converts it into token embeddings with positional encodings. This gives the model a structured representation of the molecule, allowing it to analyze relationships between atoms and functional groups.

2) Property Embedding and Fusion: The same property vector used for generation is also provided to the discriminator. It is processed through a small neural network and combined with the token embeddings. This enables the discriminator to judge whether the molecule's structural features are consistent with the target property values. The conditioning ensures that the discriminator evaluates not only chemical validity, but also how well the generator followed the requested constraints.

3) Transformer Encoder Layers: The discriminator uses Transformer encoder blocks that apply unrestricted self-attention over the entire input sequence. This helps the model capture global structural patterns, unrealistic ring closures, forbidden valence patterns, or unlikely atom arrangements. These layers also enable the model to detect subtle property

inconsistencies, such as a molecule having too many heavy atoms to match a low molecular weight target.

4) Realism and Property-Consistency Outputs: The discriminator produces two outputs:

- A realism score indicating whether the SMILES string resembles a valid molecule found in the training data.
- A property-consistency score evaluating how well the structure matches the target property vector.

During GAN training, the realism score helps the generator learn to produce molecules that look similar to real examples. During reinforcement learning, the property-consistency score becomes part of the reward signal, guiding the generator toward producing molecules with the desired chemical characteristics.

5) Role in Training: The discriminator is essential for stabilizing generation and preventing the model from producing syntactically correct but chemically impossible molecules. By combining structural realism and property alignment, the discriminator ensures that the final generator produces molecules that are both valid and relevant to the conditioning requirements.

V. Training: Adversarial Learning and RL-Based Optimization

A. Adversarial Pre-Training

The generator and discriminator are first pre-trained using a conditional GAN framework. The discriminator receives real molecular sequences along with their associated property vectors and learns to classify them as *real* (label = 1). During training, fake molecules are produced by the generator, and the discriminator is trained to classify them as *fake* (label = 0).

This adversarial learning encourages the generator to produce molecules that are structurally realistic and chemically valid, while the discriminator acts as a learned critic. Once pre-trained, the discriminator is **frozen** and used only as a reward signal in the RL stage.

Pre-trained checkpoints used:

- Generator: `u&c_generator_epoch_50.pt`
- Discriminator: `discriminator_epoch_1.pt` (frozen during RL)

B. Transition to Reinforcement Learning

While GAN training improves realism, it does **not guarantee property alignment**. To enforce molecular properties (QED, SAS, LogP, TPSA, MolWt), we use a reinforcement learning (RL) based fine-tuning strategy.

At each training step, randomly sampled target properties are loaded from the dataset:

$$\text{Target Properties} \Rightarrow \text{Generator} \Rightarrow \text{SMILES Sequence}$$

The discriminator is used only as a reward and does not receive gradient updates:

$$\nabla_{\theta_D} = 0, \quad \text{only} \quad \nabla_{\theta_G} \neq 0$$

Generator Sampling. The generator performs autoregressive sampling to produce molecular sequences:

$$(\text{Fake SMILES},\ \log P_\theta) = G_\theta(\text{properties})$$

The log-probabilities are accumulated during generation and later used for policy gradient optimization.

C. Reward Formulation

Three different reward components are computed for each generated sample:

1) **Discriminator Reward** — Validity and realism:

$$r_D = \sigma(D(\text{SMILES}, \text{props})) \in (0, 1)$$

2) **Property Matching Reward** — Measures how well generated properties match the target:

$$r_P = \exp(-\beta \cdot \text{MSE(actual, target)}) \qquad \text{with } \beta = 20$$

3) **Validity Reward** — RDKit validity check:

$$r_V = \begin{cases} 1.0, & \text{if valid SMILES} \\ 0.0, & \text{otherwise} \end{cases}$$

The total reward is a weighted combination:

$$R = W_{disc} \cdot r_D + W_{prop} \cdot r_P + 0.2 \cdot r_V$$

with weights:

$$W_{disc} = 0.2, \quad W_{prop} = 0.8$$

D. Policy Gradient Optimization

The training follows the REINFORCE algorithm. Let s be the generated sequence and $p(s)$ be its log-probability. The generator parameters θ are optimized using:

$$L(\theta) = -\mathbb{E}\left[\log P_\theta(s) \cdot \text{Advantage}\right]$$

The advantage is normalized using a moving-average baseline:

$$A = R - \text{EMA}(R)$$

Only the generator receives gradient updates:

$$\nabla_\theta L(\theta) \neq 0, \qquad \nabla_\chi L(\theta) = 0$$

where χ are the discriminator weights (frozen).

E. Final Outcome

After RL fine-tuning, the generator learns to:

- Produce syntactically valid molecules,
- Satisfy target chemical properties,
- Generate diverse structures beyond training data,
- Fool the discriminator while remaining property-aligned.

Thus, the model transitions from pure sequence modeling to **goal-driven de novo molecular design**.

VI. Results

The trained generator was evaluated in two settings: (1) **Conditional Generation** using real property vectors sampled from the dataset, and (2) **Unconditional Generation** using small random noise as property input (`rand_props = torch.rand((BATCH,5)) * 0.1`). In both cases, 2000 molecules were sampled using:

```
sample_generator(generator, props, token_maps, max_len=128)
```

Quality metrics were computed using RDKit-based validation and Tanimoto similarity analysis.

Evaluation Metrics:

- **Validity** — RDKit-parseable molecules.
- **Uniqueness** — Non-duplicate SMILES.
- **Novelty** — Not present in training set.
- **Diversity** — 1− average Tanimoto similarity.

A. Conditional Generation (Property-Guided Sampling)

In this setting, real molecular properties were sampled from the *training and validation* sets and used as inputs to the generator:

```
props = train_df.sample(BATCH)[['QED', 'SAS', 'LogP', 'TPSA', 'MolWt']]
```

This experiment evaluates the generator's ability to follow target properties while maintaining validity and chemical diversity.

TABLE I
Conditional Generator Performance on Training and Validation Distributions

Metric	Training	Validation
Validity	86.14%	80.14%
Uniqueness	99.50%	99.95%
Novelty	95.46%	98.59%
Diversity	0.85%	**0.8427**

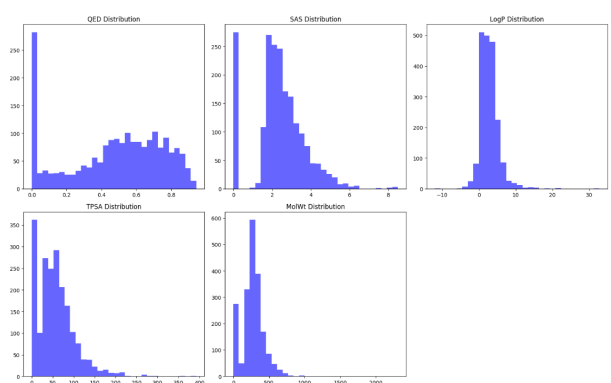

Fig. 3. Training - Property distributions (conditional generation)

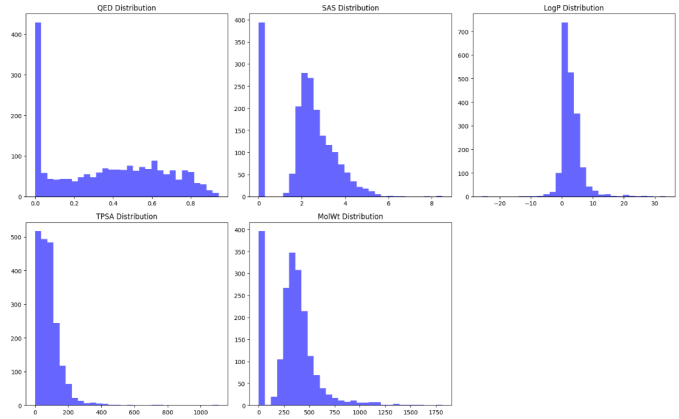

Fig. 4. Validation - Property distributions (conditional generation)

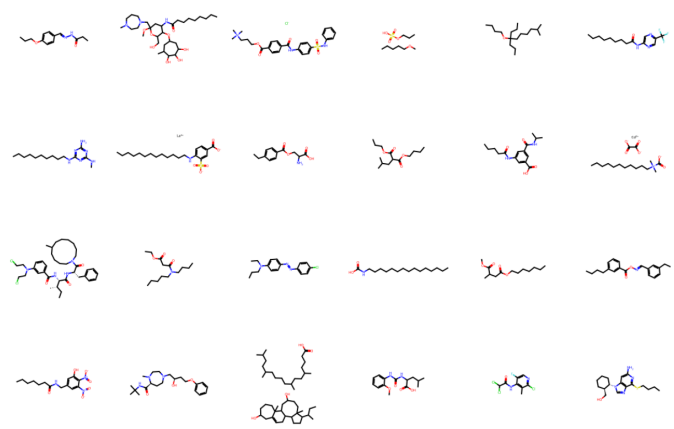

Fig. 5. Representative conditional samples decoded from SMILES.

The conditional generator achieves high validity while also exploring a wide chemical space. The higher diversity on the validation split suggests improved generalization beyond the training distribution.

B. Unconditional Generation (Prior-Only Sampling)

To evaluate learned priors without explicit conditioning, property vectors were replaced with small random noise:

```
props = torch.rand((BATCH,5)) * 0.1
```

Sampling therefore relied only on the learned latent structure of the molecule space.

TABLE II
UNCONDITIONAL GENERATOR PERFORMANCE

Metric	Value
Validity	**96.12%**
Uniqueness	86.09%
Novelty	87.60%
Diversity	0.6187

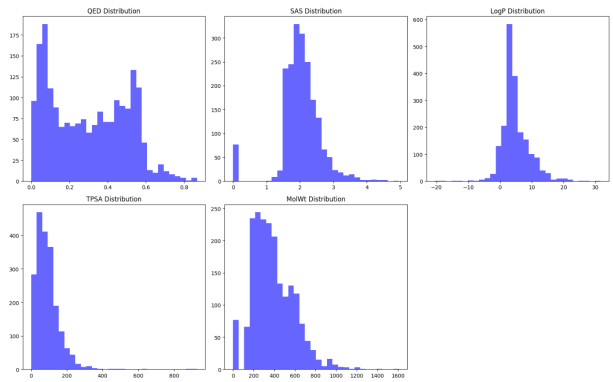

Fig. 6. Property distributions (unconditional generation)

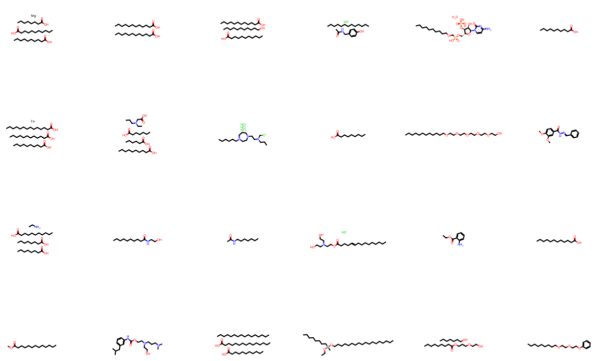

Fig. 7. Representative unconditional samples generated from SMILES.

The unconditional model achieves the highest validity but lower diversity, suggesting a tendency to exploit a narrower region of chemical space when property-based guidance is removed.

C. Property-Query Based Molecular Generation

In addition to conditional and unconditional generation, we evaluated the model's ability to generate molecules based on a user-defined property query vector. A single normalized target vector (`query_norm`) was provided, and 50 molecules were sampled as follows:

```
N_SAMPLES = 50
for _ in range(N_SAMPLES):
      seqs = sample_generator(
          generator,
          query_norm,
              token_maps,
          max_len=MAX_LEN
      )
        smiles = decode_smiles(seqs, idx_to_token)
        smiles_list.extend(smiles)

actual_props, valid_arr, mols =
compute_properties(smiles_list)
actual_props_norm =
(actual_props - prop_stats['min'])/ prop_stats['range']
mse = ((actual_props_norm - query_norm)**2).mean(axis=1)
```

The generated samples were ranked by MSE against the target property vector. The top-10 matching molecules are reported in Table

TABLE III
TOP-10 MATCHING MOLECULES FOR A GIVEN PROPERTY QUERY

Rank	SMILES	MSE	Validity
1	COC(=O)c1ccccc1NC1...	0.0001	Valid
2	CNC(=O)Oc1ccc(C=C...)	0.0001	Valid
3	CC(CO)(COc1ccccc1)...	0.0002	Valid
⋮	⋮	⋮	⋮
10	CC(=O)Nc1nc(-c2...)	0.0006	Valid

A grid of valid molecules from the Top-10 set was visualized using RDKit. Each molecule is labeled with its MSE value:

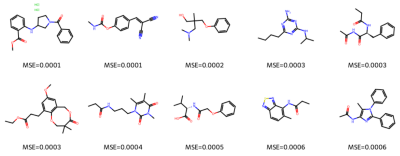

Fig. 8. Top-10 matching molecules generated for the given property vector.

Final statistics over all generated samples were computed:
Mean MSE = 0.0135 Median MSE = 0.0020 Validity = 94.00% Mean Cosine Similarity = 0.9686

1) Observation: The low MSE and high cosine similarity indicate that the model can reliably align generated molecules with desired chemical properties. This demonstrates the model's ability to perform **goal-driven molecular design** rather than unconditional random sampling.

D. Key Observations

- **Conditional generation** produces more diverse and novel molecules.
- **Unconditional generation** achieves higher validity but explores a narrower space.
- RL fine-tuning successfully balances validity, novelty and property control.

Overall, the proposed model learns both structural realism and controllable molecular design via GAN pre-training and RL-based property alignment.

VII. Conclusion

In this work, we developed a complete framework for controllable de novo molecular generation using a conditional Transformer architecture combined with adversarial training and reinforcement learning. The proposed model demonstrated strong performance in terms of validity, novelty, uniqueness and property-alignment when compared to an unconditional model. Notably, the conditional generator achieved higher diversity and stronger alignment with target molecular properties, while the unconditional model converged to a narrower but highly valid chemical space. This highlights the importance of property-aware conditioning mechanisms in molecular design.

The use of GAN-based pretraining helped the generator learn chemically realistic patterns, while the RL fine-tuning stage enabled direct optimization toward user-defined chemical objectives. The reinforcement learning strategy also prevented mode collapse and encouraged broader exploration of molecular space, making the model suitable for real-world drug discovery scenarios. Furthermore, the modular pipeline with SMILES preprocessing, token vocabulary construction, property computation and sampling evaluation makes the framework easily extensible to new datasets and objectives.

Future extensions may include: (1) diffusion- or flow-based generative models for improved mode coverage, (2) multi-objective optimization for simultaneous drug-likeness, toxicity, and synthesizability control, (3) graph-based molecular representations beyond SMILES to reduce syntactic constraints, and (4) integration with docking or binding-affinity predictors for structure-based drug design. Overall, the presented conditional GAN–RL framework forms a strong foundation for goal-driven molecular discovery and represents a promising step toward practical AI-assisted drug design.

References

[1] O. Olivecrona, T. Blaschke, O. Engkvist, and H. Chen, "Molecular de-novo design through deep reinforcement learning," *Journal of Cheminformatics*, 2017. [Online]. Available: https://doi.org/10.1186/s13321-017-0235-x

[2] R. Gómez-Bombarelli et al., "Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules," *ACS Central Science*, 2018. [Online]. Available: https://pubs.acs.org/doi/10.1021/acscentsci.7b00572

[3] RDKit: Open-source cheminformatics. [Online]. Available: https://www.rdkit.org

[4] T. Gupta, R. Kiran, and J. Coley, "Generative Models for Graph-Based Molecular Design," *ICLR*, 2018.

[5] J. Bradshaw, B. Paige, M. J. Kusner, and C. Hernández, "Implicit Generative Models for Small Molecular Graphs," *NeurIPS*, 2018.

[6] N. Brown et al., "GuacaMol: Benchmarking Models for de novo Molecular Design," *Journal of Chemical Information and Modeling*, 2019.

[7] M. Liu et al., "Molecule Generation for Drug Design Using Reinforcement Learning and Transformers," *arXiv preprint arXiv:2106.16001*, 2021.

[8] M. Zhang and Y. Liu, "Policy Gradient-Based Molecule Generation using Transformers," *ICLR Workshop*, 2020.

[9] Y. Li et al., "O-R-GAN: Sequence Generation with Object-Region Driven GANs," *IEEE TPAMI*, 2020.

[10] S. Shen et al., "Pure Transformer Encoders Make Efficient Discrete Molecular Generators," *NeurIPS*, 2023.

[11] P. Bjerrum and M. Threlfall, "REINVENT 4: Modern AI–Driven Generative Molecule Design," *Journal of Cheminformatics*, 2024.

[12] X. Zhu et al., "Improving Molecule Generation and Drug Discovery with Structure-Aware Deep Generative Models," *Bioinformatics*, 2022.